Лабораторная работа №7. CI/CD для Go-микросервиса на GitHub Actions

Цель работы: изучить принципы непрерывной интеграции и непрерывной доставки (CI/CD), освоить синтаксис GitHub Actions и настроить автоматическую сборку, тестирование и публикацию Docker-образа для Go-микросервиса.

Теоретическое обоснование

CI (Continuous Integration, непрерывная интеграция) – практика, при которой разработчики регулярно (несколько раз в день) сливают свои изменения в общую ветку. Каждое слияние автоматически проверяется: код собирается, запускаются тесты, выполняется статический анализ. Цель CI – как можно раньше обнаружить ошибки интеграции.

В классической модели с длинными ветками слияние превращается в болезненный процесс: разработчики месяцами работают в изоляции, а затем тратят недели на разрешение конфликтов. Чем дольше ветка живёт, тем дороже слияние.

Практика использования CI решает эту проблему – разработчики коммитят в main не реже раза в день. Каждый коммит проходит автоматическую проверку:

1. Код собирается (go build).
2. Запускаются unit-тесты (go test).
3. Запускаются интеграционные тесты.
4. Выполняется статический анализ (golangci-lint, go vet).
5. Проверяется покрытие тестами.
6. Проверяется безопасность зависимостей (govulncheck).

В данной практике существует **правило «зелёного main»**, которое гласит, что ветка main всегда должна быть в рабочем состоянии. Если сборка упала – её чинят в первую очередь, до всех остальных задач. Это правило называется «stop the line» (по аналогии с конвейером Toyota).

Инструменты CI: GitHub Actions, GitLab CI, Jenkins, CircleCI, Travis CI, Buildkite, Drone.

CD (Continuous Delivery / Continuous Deployment)

CD в контексте разработки программного обеспечения означает Continuous Delivery (непрерывная доставка) и тесно связанный с ней термин Continuous Deployment (непрерывное развертывание). Это практики автоматизации выпуска обновлений кода в тестовые или боевые среды.

Continuous Delivery – каждое изменение, прошедшее CI, автоматически собирается в артефакт (Docker-образ, бинарник) и готово к развёртыванию. Но развёртывание в production требует ручного подтверждения (нужно нажать кнопку «deploy»).

Continuous Deployment – развёртывание в production происходит автоматически после успешного прохождения всех проверок. Не требует человека в цепочке.

Разница между ними – в наличии ручного шага подтверждения.

Компоненты CD:

– Artifact repository — хранилище артефактов (Docker Hub, GitHub Container Registry, Nexus, Artifactory);

– Environment — целевое окружение (dev, staging, production);

– Deployment strategy — способ выката.

Стратегии развёртывания

При выпуске новой версии в production важно минимизировать риск. Основные стратегии:

1. Recreate (пересоздание).

Останавливаем старую версию, запускаем новую. Простой в момент переключения. Используется редко (только для dev-сред).

2. Rolling update (постепенное обновление).

Постепенно заменяем экземпляры старой версии на новые. Нет простоя, но во время выката сосуществуют обе версии. Стандарт в Kubernetes.

3. Blue-green (сине-зелёный).

Держим два идентичных окружения: blue (текущее) и green (новое). После выката в green — переключаем трафик. Мгновенный откат — просто вернуть трафик на blue.

4. Canary (канареечный).

Направляем небольшой процент трафика (например, 5%) на новую версию. Если метрики в норме — увеличиваем до 100%. Если нет — откатываем. Название — от «канарейки в шахте» (Выражение «канарейка в шахте» (англ. miner's canary) — это исторический факт, который стал идиомой. Он означает ранний маркер или индикатор скрытой опасности, которая угрожает всей системе).

5. A/B-тестирование.

Похоже на canary, но трафик направляется по признаку пользователя (например, по региону). Используется для проверки гипотез, а не для выката.

Инструменты CD: GitLab CI/CD и GitHub Actions, Argo CD, Jenkins, CircleCI, TeamCity.

Классический CI/CD-конвейер (pipeline) состоит из этапов:



Основные этапы CD-пайплайна:

– Сборка (Build): превращение исходного кода в готовый артефакт или контейнер (например, через Docker);

– Тестирование (Testing): запуск автоматических тестов (модульных, интеграционных, проверки безопасности);

– Публикация артефакта (Artifact Registry): сохранение сборки в репозиторий;

– Развертывание в Staging: выкатка на тестовый сервер, имитирующий боевой;

– Развертывание в Production: попадание в реальную среду (ручное в Delivery или автоматическое в Deployment).

Зачем это DevOps-специалисту

Без CI/CD:

– релизы выпускаются вручную, долго и с ошибками;

– тесты запускаются «когда вспомнят»;

– сборка Docker-образа зависит от локальной машины разработчика («у меня работает»).

С CI/CD:

– каждое изменение автоматически проверяется;

– артефакты собираются в едином окружении;

– результат воспроизводим и задокументирован.

Обзор инструментов CI/CD

Инструмент	Тип	Особенности
GitHub Actions	SaaS	Встроен в GitHub, YAML, marketplace actions
GitLab CI	SaaS/Self-hosted	Встроен в GitLab, .gitlab-ci.yml, runners
Jenkins	Self-hosted	Гибкий, плагины, но требует администрирования
CircleCI	SaaS	Быстрый, удобный UI, orbs
Travis CI	SaaS	Простой, исторически популярен в open source
Drone	Self-hosted	Контейнерный, простой
TeamCity	Self-hosted	От JetBrains, мощный UI
Buildkite	Гибрид	Агенты self-hosted, управление SaaS

GitHub Actions

GitHub Actions – встроенная в GitHub платформа CI/CD. Она позволяет описывать конвейеры в виде YAML-файлов прямо в репозитории.

Основные компоненты:

Workflow. YAML-файл в .github/workflows/. В одном репозитории может быть несколько workflow. Автоматизированный процесс, описывается в .github/workflows/*.yml.

Структура workflow-файла (файл .yaml):

```yaml
name: CI # имя workflow (отображается в UI GitHub)

on: # события-триггеры
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs: # описание job'ов
  build:
    runs-on: ubuntu-latest # runner
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Go
        uses: actions/setup-go@v5
        with:
          go-version: '1.22'

      - name: Build
        run: go build -v ./...
```

Event. Событие, запускающее workflow. Основные:

Event	Описание
push	Push в ветку или тег
pull_request	Открытие, синхронизация, реопен PR
schedule	Cron-расписание
workflow_dispatch	Ручной запуск из UI
release	Публикация релиза
issues	События с issues
workflow_call	Вызов из другого workflow

Job. Набор шагов, выполняемых на одном runner'e. Job'ы по умолчанию выполняются параллельно. Зависимости задаются через needs:.

Step. Отдельная задача внутри job. Может быть:

– run: — команда shell;

– uses: — вызов готового action.

Action. Переиспользуемый блок. Бывает трёх видов:

– Docker action — выполняется в контейнере;

– JavaScript action — на Node.js;

– Composite action — набор шагов, объединённых в один.

Runner. Виртуальная машина, на которой выполняется job (ubuntu-latest, windows-latest, macos-latest).

GitHub предоставляет:

– ubuntu-latest (Ubuntu 22.04);

– ubuntu-22.04, ubuntu-24.04;

– windows-latest;

– macos-latest.

Также можно запустить self-hosted runner — свою машину.

Contexts и выражения

В workflow доступны контексты — объекты с информацией о запуске:

Context	Содержит
github	Информация о репозитории, событии, акторе
env	Переменные окружения workflow
secrets	Секреты репозитория/окружения
matrix	Значения матрицы
needs	Результаты зависимых job'ов
steps	Результаты предыдущих шагов
runner	Информация о runner'е
job	Информация о текущем job'е

Синтаксис: ${{ context.property }}. Примеры:

```
${{ github.repository }}          # owner/repo
${{ github.sha }}                 # полный SHA коммита
${{ github.ref }}                 # refs/heads/main
${{ github.event_name }}          # push, pull_request, ...
${{ secrets.DOCKERHUB_TOKEN }}    # секрет
${{ matrix.go-version }}          # значение матрицы
```

```
${{ needs.build.outputs.version }}# output из job build
```

Функции: hashFiles(), format(), contains(), startsWith(), endsWith(), join(), toJSON().

Secrets, variables, environments, artifact

Secrets — зашифрованные значения, недоступны для чтения. Задаются на трёх уровнях:

– Repository secrets — доступны всем workflow репозитория;

– Environment secrets — привязаны к окружению (staging, production);

– Organization secrets — общие для всех репозиториев организации.

Секреты задаются в GitHub: Settings → Secrets and variables → Actions. В workflow они доступны как ${{ secrets.NAME }}. Примеры:

DOCKERHUB_USERNAME – логин Docker Hub;

DOCKERHUB_TOKEN – токен доступа.

Секреты никогда не должны попадать в код или в логи.

Variables — незашифрованные значения (например, версия Go, имя образа). Доступны как ${{ vars.NAME }}.

Environments — целевые окружения с собственными секретами и правилами защиты. Могут требовать approve перед деплоем.

Artifact – файл, сохраняемый после выполнения job (например, бинарник, лог).

Кэширование

Сборка Go-проекта может занимать минуты из-за скачивания зависимостей. GitHub Actions позволяет кэшировать каталоги между запусками. Для Go:

Каталог	Что кэшировать
~/go/pkg/mod	Скачанные модули
~/.cache/go-build	Результаты сборки

Ключ кэша должен включать хэш go.sum, чтобы кэш сбрасывался при изменении зависимостей.

Стратегия кэширования:

– key — точный ключ (например, по хэшу go.sum);

– restore-keys — резервные ключи для частичного совпадения.

Кэширование происходит через actions/cache:

```
- uses: actions/cache@v4
  with:
    path: ~/go/pkg/mod
    key: ${{ runner.os }}-go-${{ hashFiles('**/go.sum') }}
```

Ключ кэша (key) включает хэш go.sum – если зависимости изменились, кэш сбросится.

Матричные сборки

Если нужно тестировать на нескольких версиях Go или ОС, используется strategy.matrix:

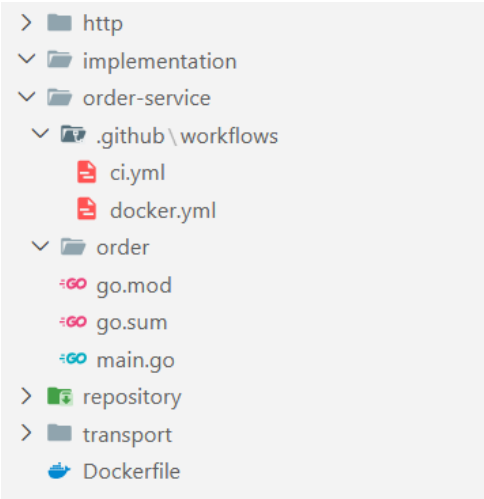
```
strategy:
  matrix:
    go-version: ['1.21', '1.22']
    os: [ubuntu-latest, macos-latest]
```

GitHub создаст отдельный job для каждой комбинации.

Практическая часть

В этой работе мы настроим CI/CD для Go-микросервиса, созданного в лабораторной работе 6. Предполагается, что проект лежит в репозитории GitHub.

Структура репозитория:



ci.yml – сборка + тесты.

docker.yml – сборка и публикация Docker-образа.

Шаг 1. Подготовка репозитория

Убеждаемся, что проект из лабораторной работы 6 загружен в GitHub.

GitHub Actions работают только внутри репозитория GitHub. Если проект ещё не в GitHub – нужно его запушить (см. лабораторную работу 5).

Это также важно для последующих шагов: branch protection, секреты, артефакты – всё это настраивается на уровне репозитория GitHub.

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/<username>/order-service.git
git push -u origin main
```

Итог шага: открыв репозиторий на github.com, вы видите файлы проекта в ветке main.

Шаг 2. Первый workflow: сборка и тесты (ci.yml)

Создаём файл .github/workflows/ci.yml, который будет запускаться при каждом push и pull request в ветку main.

Это базовый CI. Любое изменение кода должно:

1. Собираться (компилироваться).

2. Проходить тесты.

3. Проверяться линтером.

Если хотя бы один из этих этапов падает — разработчик сразу видит проблему и исправляет её до мержа.Файл ci.yml:

Создайте каталог и файл:

```
mkdir -p .github/workflows
touch .github/workflows/ci.yml
```

Вставьте содержимое:

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    name: Build and Test
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Go
        uses: actions/setup-go@v5
        with:
          go-version: '1.22'

      - name: Cache Go modules
        uses: actions/cache@v4
        with:
          path: ~/go/pkg/mod
          key: ${{ runner.os }}-go-${{ hashFiles('**/go.sum') }}
          restore-keys: |
            ${{ runner.os }}-go-

      - name: Download dependencies
        run: go mod download

      - name: Verify dependencies
```

```
      run: go mod verify

    - name: Build
      run: go build -v ./...

    - name: Run tests
      run: go test -v -race -coverprofile=coverage.out ./...

    - name: Upload coverage artifact
      uses: actions/upload-artifact@v4
      with:
        name: coverage
        path: coverage.out
```

Пояснения к коду:

– name: CI — имя workflow. Оно отображается во вкладке Actions GitHub. Может быть любым, но лучше осмысленным.

– on: — блок триггеров. У нас два:

– push.branches: [main] — workflow запускается при push в main.

– pull_request.branches: [main] — и при открытии/обновлении PR в main.

– jobs: — секция описания job'ов. У нас один job build-and-test.

– runs-on: ubuntu-latest — тип runner'а. GitHub предоставляет бесплатные runner'ы: ubuntu-latest, windows-latest, macos-latest. Для Go-проектов почти всегда используется Ubuntu — он быстрее и дешевле.

– steps: — пошаговый список. Шаги выполняются последовательно: если один упал, следующие не выполняются.

– actions/checkout@v4 — стандартный action, клонирующий ваш репозиторий в рабочую директорию runner'а. Без него go build не найдёт файлы.

– actions/setup-go@v5 — устанавливает Go нужной версии. Параметр go-version: '1.22' указывает, какую именно версию скачать.

– actions/cache@v4 — кэширует ~/go/pkg/mod (каталог с модулями). Ключ формируется из ОС и хэша go.sum. Если go.sum не менялся — кэш переиспользуется, и go mod download выполняется быстрее.

– go mod download — скачивает зависимости.

– go mod verify — проверяет, что скачанные модули не повреждены.

– go build -v ./... — собирает все пакеты проекта. Флаг -v — verbose.

– go test -v -race -coverprofile=coverage.out ./... — запускает тесты. Флаг -race включает детектор гонок (важно для параллельного кода), -coverprofile=coverage.out записывает профиль покрытия в файл.

– actions/upload-artifact@v4 — сохраняет coverage.out как артефакт. Его можно скачать после выполнения workflow.

Итог шага: в репозитории появился файл .github/workflows/ci.yml. При push workflow автоматически запускается.

Шаг 3. Линтинг: golangci-lint

Добавляем шаг статического анализа.

Линтер находит потенциальные ошибки до того, как код попадёт в main:

– неиспользуемые переменные;

– забытые return;

– необработанные ошибки;

– гонки;

– проблемы стиля.

golangci-lint — стандарт де-факто: он объединяет десятки линтеров в одном инструменте.Добавьте в ci.yml перед шагом Build:

```
      - name: Run golangci-lint
        uses: golangci/golangci-lint-action@v6
        with:
          version: v1.60
          args: --timeout=5m
```

Пояснение к коду:

– golangci/golangci-lint-action@v6 — официальный action.

– version: v1.60 — версия линтера. Лучше указывать явно, чтобы обновление линтера не сломало сборку неожиданно.

– --timeout=5m — таймаут; по умолчанию 1 минута, этого может не хватить на большом проекте.

Итог шага: линтер проходит — зелёная галочка в логах.

Шаг 4. Матричная сборка на нескольких версиях Go

Меняем job так, чтобы он запускался на нескольких версиях Go.

Go выпускает новые версии каждые полгода. Матрица позволяет убедиться, что код работает на обеих поддерживаемых версиях (текущей и предыдущей). Это критично для библиотек и публичных проектов, но полезно и для внутренних сервисов.

Замените блок jobs: целиком:

```
jobs:
  build-and-test:
    name: Build and Test (Go ${{ matrix.go-version }})
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        go-version: ['1.21', '1.22']

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-go@v5
        with:
          go-version: ${{ matrix.go-version }}

      - name: Download dependencies
        run: go mod download

      - name: Build
        run: go build -v ./...
```

```
      - name: Test
        run: go test -v -race ./...
```

Пояснения:

– strategy.matrix.go-version: ['1.21', '1.22'] — GitHub создаст два job'a: один для Go 1.21, второй для Go 1.22.

– name: Build and Test (Go ${{ matrix.go-version }}) — имя job'a динамическое. В UI вы увидите «Build and Test (Go 1.21)» и «Build and Test (Go 1.22)».

– fail-fast: false — если один job упал, второй всё равно выполняется. По умолчанию GitHub отменяет остальные job'ы при первой ошибке; для матрицы это неудобно, потому что хочется видеть результаты по всем версиям.

Итог шага: во вкладке Actions отображаются два job'a в одном workflow.

Шаг 5. Dockerfile для сервиса

Чтобы CI мог собрать Docker-образ, нужен Dockerfile. Подробнее про Docker будет рассматриваться в дальнейших работах, здесь – минимально необходимый вариант.

Создайте файл Dockerfile в корне проекта:

```
# === Этап 1: сборка ===
FROM golang:1.22-alpine AS builder

WORKDIR /app

# Копируем go.mod и go.sum отдельно – так кэш слоёв Docker
# переиспользуется, если изменился только код
COPY go.mod go.sum ./
RUN go mod download

COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o /app/server .
```

```
# === Этап 2: рантайм ===
FROM alpine:3.20

RUN apk --no-cache add ca-certificates

WORKDIR /root/

COPY --from=builder /app/server .

EXPOSE 8080

ENTRYPOINT ["./server"]
```

Пояснения:

– Multi-stage build — сначала собираем в тяжёлом образе golang:1.22-alpine (~300 МБ), потом копируем бинарник в лёгкий alpine:3.20 (~5 МБ). Итоговый образ весит ~10 МБ вместо ~300 МБ.

– CGO_ENABLED=0 — статическая сборка. Бинарник не зависит от системных библиотек, его можно запускать в scratch.

– GOOS=linux — явно указываем целевую ОС.

– COPY go.mod go.sum ./ отдельно от COPY . . — Docker кэширует слои. Если код изменился, но go.mod нет — go mod download не выполнится заново.

– COPY --from=builder /app/server . — копируем бинарник из первого этапа.

– EXPOSE 8080 — документируем порт (не открывает его, а служит подсказкой).

– ENTRYPOINT ["./server"] — команда запуска контейнера.

Итог шага: файл Dockerfile в корне проекта.

Шаг 6. Workflow для сборки и публикации Docker-образа (docker.yml)

Создаём отдельный workflow, который собирает Docker-образ и пушит его в Docker Hub. Это и есть CD: артефакт (образ) автоматически готовится и публикуется после успешного CI. Образ доступен для скачивания, его можно развернуть в Kubernetes (будет рассмотрено в дальнейших работах).

Создайте .github/workflows/docker.yml:

```
name: Docker

on:
  push:
    branches: [main]
    tags: ['v*']
  pull_request:
    branches: [main]

env:
  REGISTRY: docker.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  docker:
    name: Build and Push Docker Image
    runs-on: ubuntu-latest

    permissions:
      contents: read
      packages: write

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3
```

```
      - name: Log in to Docker Hub
        if: github.event_name != 'pull_request'
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Extract metadata
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
          tags: |
            type=ref,event=branch
            type=ref,event=pr
            type=semver,pattern={{version}}
            type=sha,prefix=sha-

      - name: Build and push
        uses: docker/build-push-action@v6
        with:
          context: .
          push: ${{ github.event_name != 'pull_request' }}
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

Пояснения:

– on.push.tags: ['v*'] — workflow запускается не только на push в main, но и при push тега, начинающегося с v (например, v1.0.0). Это позволяет делать релизы.

– env.REGISTRY: docker.io — глобальная переменная окружения. IMAGE_NAME: ${{ github.repository }} — имя образа совпадает с полным именем репозитория (username/order-service).

– permissions: — права токена GitHub Actions. contents: read — чтение кода, packages: write — запись в GitHub Packages (если понадобится).

– docker/setup-buildx-action@v3 — устанавливает Buildx — расширенный сборщик Docker, поддерживающий multi-arch и продвинутое кэширование.

– docker/login-action@v3 — логин в Docker Hub. if: github.event_name != 'pull_request' — при PR логин не выполняется, потому что секреты недоступны для PR из форков (мера безопасности GitHub).

– docker/metadata-action@v5 — автоматически формирует теги. Например:

– type=ref,event=branch → main;

– type=ref,event=pr → pr-123;

– type=semver,pattern={{version}} → 1.0.0 (при push тега v1.0.0);

– type=sha,prefix=sha- → sha-abc1234 (короткий SHA коммита).

– docker/build-push-action@v6 — собирает и пушит образ.

– push: ${{ github.event_name != 'pull_request' }} — при PR образ только собирается, но не пушится.

– cache-from / cache-to: type=gha — кэш слоёв Docker между запусками GitHub Actions.

– context: . — контекст сборки — корень репозитория (там лежит Dockerfile).

Итог шага: Workflow собирается и пушит образ. В Docker Hub появится образ username/order-service с тегами.

Шаг 7. Настройка секретов

Добавляем в GitHub секреты для Docker Hub.

Пароли и токены нельзя хранить в коде. Если положить токен в docker.yml в открытом виде — злоумышленник получит доступ к вашему Docker Hub. Секреты GitHub шифруются и доступны только во время выполнения workflow.

1. Создайте на Docker Hub Access Token: Account Settings → Security → New Access Token.

2. В GitHub откройте Settings → Secrets and variables → Actions → New repository secret.

3. Добавьте:

DOCKERHUB_USERNAME – ваш логин;

DOCKERHUB_TOKEN – созданный токен.

Итог шага: в настройках репозитория появится два секрета.

Шаг 8. Проверка работы

Делаем коммит и push, наблюдаем за выполнением workflow.

Это финальная проверка: все компоненты собраны вместе, и мы убеждаемся, что CI/CD работает end-to-end.

```
git add .github/workflows/ci.yml .github/workflows/docker.yml Dockerfile
git commit -m "Add CI/CD pipelines"
git push
```

Алгоритм действи:

1. Откройте вкладку Actions в репозитории GitHub.

2. Убедитесь, что оба workflow запустились.

3. Дождитесь завершения.

4. Проверьте, что образ появился в Docker Hub.

Что должно получится после выполнения этого шага:

1. Вкладка Actions показывает два workflow: CI и Docker.

2. Оба запущены и выполняются.

3. По завершении — зелёные галочки.

4. В Docker Hub — новый образ.

Шаг 9. Бейдж статуса сборки

Добавляем в README.md бейдж статуса CI. Бейдж показывает всем посетителям репозитория, что сборка проходит. Это стандарт для open-source и внутренних проектов.

В README.md добавьте:

![CI](https://github.com/<username>/order-service/actions/workflows/ci.yml/badge.svg)

![Docker](https://github.com/<username>/order-service/actions/workflows/docker.yml/badge.svg)

Пояснение. URL формируется по шаблону:

https://github.com/<owner>/<repo>/actions/workflows/<file>.yml/badge.svg

Итог шага: в README отображается зелёный бейдж «CI passing».

Шаг 10. Защита ветки main

Настраиваем branch protection.

Запрещаем мерж в main, если CI не прошёл. Это делает CI обязательным, а не «декоративным». Без защиты можно случайно смержить сломанный код, и весь смысл CI теряется.

1. Settings → Branches → Add branch protection rule.
2. Branch name pattern: main.
3. Включите:
   – Require a pull request before merging;
   – Require status checks to pass before merging;
   – Выберите job'ы Build and Test (Go 1.21), Build and Test (Go 1.22).

Итог шага: прямой push в main запрещён.

Задания

Задание 1. Базовое

1. Загрузить проект из ЛР 6 в GitHub.

2. Создать .github/workflows/ci.yml со сборкой и тестами.

3. Убедиться, что workflow запускается на push и PR.

4. Посмотреть логи выполнения, найти шаги Build и Test.

5. Добавить бейдж статуса в README.md.

Задание 2. Матрица и линтинг

1. Добавить матричную сборку на Go 1.21 и 1.22.

2. Добавить шаг golangci-lint.

3. Убедиться, что оба job'а проходят.

4. Сделать коммит с намеренной ошибкой (например, неиспользуемая переменная) и убедиться, что линтер его поймал.

Задание 3. Docker-образ

1. Написать Dockerfile для своего сервиса (multi-stage build).

2. Создать .github/workflows/docker.yml.

3. Настроить секреты DOCKERHUB_USERNAME и DOCKERHUB_TOKEN.

4. Убедиться, что образ пушится в Docker Hub.

5. Скачать образ и запустить контейнер локально, проверить curl-запросы.

Задание 4. Защита ветки

1. Настроить branch protection для main.

2. Создать тестовую ветку, сделать в ней PR.

3. Убедиться, что мерж заблокирован, пока CI не прошёл.

4. Исправить ошибку, дождаться зелёного CI, смержить PR.

Задание 5. Кэширование и артефакты

1. Добавить кэш ~/go/pkg/mod в ci.yml.

2. Убедиться, что второй запуск workflow выполняется быстрее первого (посмотреть время в логах).

3. Сохранить coverage.out как artifact.

4. Скачать artifact и открыть локально (go tool cover -html=coverage.out).

Варианты индивидуальных заданий

Индивидуализация для работы идёт не по домену, а по типу пайплайна. Каждый вариант – набор требований к CI/CD.

Вариант 1. Базовая матрица

CI на Go 1.21 и 1.22, линтер, тесты с покрытием, artifact coverage.out.

Вариант 2. Docker + Docker Hub

CI + сборка образа с multi-stage build + push в Docker Hub с тегами main и sha-<short>.

Вариант 3. Релизы по тегам

При push тега v* – сборка бинарников под три ОС и создание GitHub Release.

Вариант 4. Pull Request Check

Проверка PR: сборка, тесты, линтер, комментарий с покрытием в PR.

Вариант 5. Кэширование

Активное использование actions/cache для ~/go/pkg/mod и ~/.cache/go-build. Замер ускорения.

Вариант 6. Безопасность

Добавить шаги: go vet, govulncheck (поиск уязвимостей в зависимостях), gosec (статический анализ безопасности).

Вариант 7. Scheduled runs

Workflow по расписанию (on.schedule: cron) – ежедневная проверка сборки на устаревших версиях Go.

Вариант 8. Multi-arch образ

Сборка Docker-образа под linux/amd64 и linux/arm64 через docker/setup-qemu-action и Buildx.

Вариант 9. Deploy в Kubernetes

После успешной сборки образа – деплой в kind-кластер на runner'е через kubectl apply.

Вариант 10. Уведомления

Отправка статуса сборки в Telegram и/или Slack через соответствующие actions.

Вариант 11. Сравнение производительности

Добавить бенчмарки (go test -bench=.) и сравнение результатов между двумя версиями Go.

Вариант 12. Мониторинг pipeline

Добавить шаг публикации метрик pipeline в Prometheus (например, через pushgateway).

Вариант 13. Монорепозиторий

Организовать CI для репозитория с несколькими сервисами: запускать сборку только того сервиса, чьи файлы изменились (через paths: фильтр и dorny/paths-filter).

Вариант 14. Секреты и окружения

Использовать GitHub Environments (staging, production) с разными секретами. Деплой в staging – автоматически, в production – с ручным подтверждением.

Вариант 15. Полный пайплайн

Собрать всё вместе: матрица, кэш, линтер, тесты с покрытием, security-скан, Docker build+push, GitHub Release, уведомление в Telegram.